

Mental Math Progression & Level System (v1)

1 Vision

- No global Elo system.
- Personalized progression per user.
- Each level corresponds to a **pattern family**.
- Progression based on:
 - Relative speed (time-to-first-correct; median vs population)
 - Skip behavior
- Long-term target: remove hard-coded thresholds and use population-adaptive percentiles.

In v1, cutoffs are hard-coded for simplicity; later versions can fully adapt automatically using population response-time percentiles.

2 High-Level Structure

Progression is separated by **Operation Domains**:

- Addition (ADD)
- Multiplication (MUL)
- Subtraction (SUB)
- Division (DIV)

Each domain has its own independent level ladder.
User progress is tracked per domain.

3 Pattern-Based Levels

Each level corresponds to a defined structural pattern.

A pattern is **not** a single question. It is a generator rule with structural constraints.
Each pattern contains:

- Operation
- Digit configuration
- Carry constraints (operation-specific)
- Other structural constraints (pattern-specific)

Operand Order Randomization

For all patterns involving operands of different digit lengths, the order of the operands is randomized at generation time (e.g., 1-digit + 2-digit or 2-digit + 1-digit). For equal-length operands (e.g., 1-digit + 1-digit), order is naturally symmetric.

Example Patterns (Addition) — first 5 levels

- Level 1: 1-digit + 1-digit, no carry
- Level 2: 1-digit + 1-digit, with carry
- Level 3: 1-digit + 2-digit, no carry
- Level 4: 1-digit + 2-digit, with carry
- Level 5: 2-digit + 2-digit, no carry

Example Patterns (Multiplication) — first 5 levels

- Level 1: 1-digit $\times$ 1-digit, no carry
- Level 2: 1-digit $\times$ 1-digit, with carry
- Level 3: 1-digit $\times$ 2-digit, no carry
- Level 4: 1-digit $\times$ 2-digit, with carry
- Level 5: 1-digit $\times$ 2-digit, two carries

Example Patterns (Subtraction) — first 5 levels

- Level 1: 1-digit $-$ 1-digit, non-negative result (no borrow possible)
- Level 2: 2-digit $-$ 1-digit, no borrow
- Level 3: 2-digit $-$ 1-digit, with borrow (single borrow)
- Level 4: 2-digit $-$ 2-digit, no borrow
- Level 5: 2-digit $-$ 2-digit, with borrow (single borrow)

Example Patterns (Division) — first 5 levels

- Level 1: 1-digit $\div$ 1-digit, 1-digit quotient
- Level 2: 2-digit $\div$ 1-digit, 1-digit quotient
- Level 3: 2-digit $\div$ 1-digit, 2-digit quotient
- Level 4: 2-digit $\div$ 2-digit, 1-digit quotient
- Level 5: 3-digit $\div$ 2-digit, 1-digit quotient

4 Performance Model (v1: Hard-Coded Cutoff)

Progression is based on:

- Whether the attempt is solved (correct answer submitted within C_L milliseconds)
- If time-to-first-correct exceeds C_L , the attempt counts as unsolved regardless of correctness

For v1, each (domain, level) has a hard-coded **cutoff time** C_L .

This threshold is tuned manually and can later be replaced by population percentiles.

5 Promotion Rule (Per Level)

For a user currently on Level L in a domain, evaluate the last N completed attempts at Level L .

$$N = 5$$

Promotion requires:

1. All of the last N attempts are solved (correct and within C_L)

If satisfied, unlock Level $L + 1$.

6 Demotion Rule (Hysteresis)

Evaluate the last M attempts at the current level (including skips).

$$M = 10$$

Demote if any of the following holds:

- Unsolved rate over the last M attempts exceeds tolerance S_{max}
- $S_{max} = 0.25$ (25%)

Only allow demotion after a minimum exposure at that level (10 attempts).

Soft Demotion Target

Demotion is intentionally soft:

- If the user is at level L , a demotion moves to $L - 1$ (minimum level is 1).
- This is true even if the triggering failures happened on a sampled lower level $k < L$.
- Example: if user is at level 10 and repeated failures are observed on level 8 questions, demotion target is level 9 (not level 7).

Formally:

$$L_{new} = \max(1, L - 1)$$

7 Level Selection During Play

Let the user's current level in a domain be L . During play, sample question level k from the set:

$$k \in \{1, 2, \dots, L, L + 1\}$$

Use fixed top-level buckets:

$$p(L) = 0.50, \quad p(L + 1) = 0.10$$

The remaining mass is assigned to all lower levels:

$$\sum_{k=1}^{L-1} p(k) = 0.40$$

Distribute this 0.40 over levels $1..L - 1$ with geometric taper:

$$u(k) = r^{(L-1)-k}, \quad 1 \leq k \leq L - 1, \quad 0 < r < 1$$

$$p(k) = 0.40 \cdot \frac{u(k)}{\sum_{j=1}^{L-1} u(j)}, \quad 1 \leq k \leq L - 1$$

$$\text{If } L = 1 : \quad p(1) = 0.90, \quad p(2) = 0.10$$

So the current level is always most likely, the next level is fixed at 10%, and all lower levels remain possible with decreasing probability.

Symbol definitions

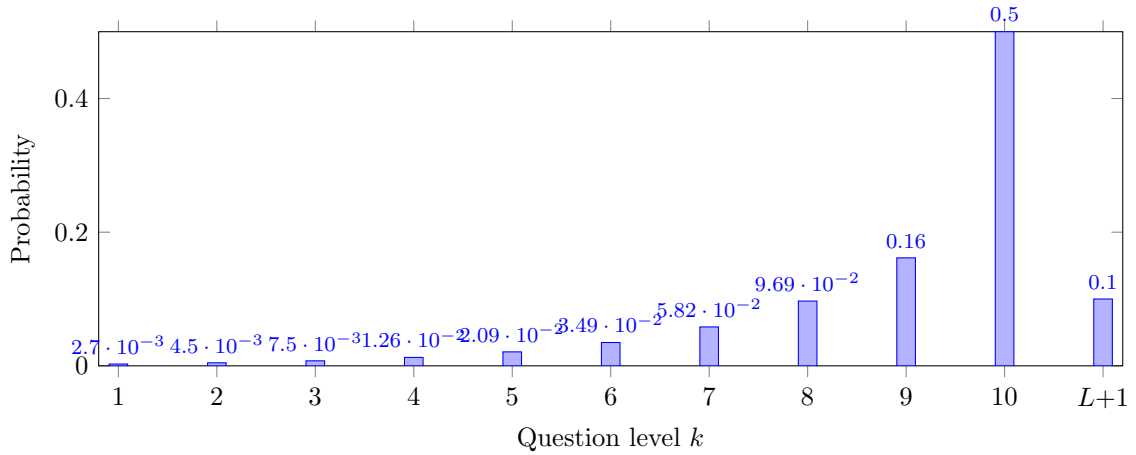
- L : current user level in the domain
- k : sampled question level
- $r \in (0, 1)$: geometric decay factor for lower levels (smaller r means faster taper)
- $u(k)$: unnormalized geometric weight for lower levels

Stability after demotion

The level sampling rule above is always the same, regardless of why a demotion happened. After demotion, only L changes; the same weighting formula and hyperparameters are reused.

Example distribution (bar chart)

Example with $L = 10$ and $r = 0.6$ (final $p(k)$):



Within a level:

- Randomize seeds
- Randomize digit combinations
- Maintain structural constraints

8 How Questions are stored

1. Store **Pattern** rows (the generator rules).
2. Generate question instances on demand using a RNG seed.
3. Store each user interaction as an **Attempt** row.

So, in practice, the “question” is represented by:

(patternId, seed)

Optional later: add a **QuestionInstance** table only if you need explicit review queues or caching.

How a Pattern is stored in DB

Each pattern family is one row in the **pattern** table.

Stable, indexed fields are stored as normal columns:

- **id**: unique pattern id
- **domain**: ADD | MUL | SUB | DIV
- **level**: ladder level within domain
- **description**: human-readable pattern label
- **cutoffTimeMs**: solved-time cutoff for v1
- **active**: enable/disable pattern
- **createdAt, updatedAt**: audit timestamps

Dynamic generation constraints are stored in:

- **params** (Prisma Json, PostgreSQL JSONB)

Uniqueness rule in v1:

`UNIQUE(domain, level, description)`

What params JSON looks like

In v1, digit fields are stored as single integers (not lists), because each pattern uses one fixed digit size per side.

ADD example (Level 3: 1-digit + 2-digit, no carry)

```
{
  "operation": "ADD",
  "leftDigits": 1,
  "rightDigits": 2,
  "carryCount": 0,
}
```

MUL example (Level 5: 1-digit x 2-digit, two carries)

```
{
  "operation": "MUL",
  "leftDigits": 1,
  "rightDigits": 2,
  "carryCount": 2,
}
```

SUB example (Level 5: 2-digit - 2-digit, single borrow)

```
{
  "operation": "SUB",
  "leftDigits": 2,
  "rightDigits": 2,
  "borrowCount": 1,
  "allowNegative": false
}
```

DIV example (Level 4: 2-digit / 2-digit, exact integer)

```
{
  "operation": "DIV",
  "dividendDigits": 2,
  "divisorDigits": 2,
  "exactInteger": true,
  "quotientDigits": 1
}
```

The exact key set may vary by operation and version. The important rule is: keep frequently queried fields in columns and operation-specific constraints in **params**.